# Appendix

## Table of Contents

## A More Implementation Details

In this section, our goal is to provide the details of system implementation (mentioned in Section 4.2) and experiment settings (mentioned in Section 5), as well as the pseudocode.

**System Details.** We implement $H_2O$ on top of FlexGen. FlexGen is a white-box implementation of OPT models, and we have done some surgery on handling the KV cache. Specifically, for the given parameter $K$, we always maintain a list of KV cache with the first $K$ entries as heavy hitters, and the last $K$ entries as most recent tokens. In order to avoid data movement in memory, the memory for KV cache is preallocated. We use a circular queue to update the last $K$ entries efficiently.

**Experiment Details.** Our study involves the evaluation with varying sizes of KV cache that encompass $4\%$, $10\%$, $20\%$, and $60\%$ of the prompt's length. We select two tasks (XSUM and CNN/Daily Mail) from the HELM framework [16] and present the performance based on $1000$ test samples. Additionally, we employ the lm-eval-harness framework [15] for six other tasks (COPA, MathQA, OpenBookQA, PiQA, RTE, and Winogrande). For the tasks derived from the HELM framework, we report the performance for zero-shot inference, while for the tasks from lm-eval-harness, we default to conduct five-shot inference.

**Pseudocode.** We show a simplified pseudocode below to demonstrate our implementation skeleton. The function `generation_loop()` is the base loop in FlexGen that controls prefetch and overlap the I/O streams and computation. Then in function `compute_layer()`, the function `attention_forward()` will be called for attention layers. During prefill iterations, the function `compute_attention()` will return $K$ heavy hitters and $K$ recent tokens if the prompt has a length greater than or equal to $2K$, otherwise return all the KV cache. The function `compute_attention()` will return new KV cache and the indices for evicted entries during decoding iterations, which would be the place to implement a customized eviction strategy. (If the current number of stored KV cache is less than $2K$, the eviction will be ignored.) During each decoding iteration, the oldest one among the last $K$ tokens (if we have no less than $K$ tokens' KV cache stored) will be removed from the reserved last $K$ entries, one heavy hitter will be evicted for each head, and the newest token will be added to the KV cache with a position in the last $K$ entries. This happens in the function `store_cache()`.

```
def generation_loop(...):
  # Prologue
  ...
  # Generate
  for i in range(gen_len):
    for j in range(num_layers):
      for k in range(num_gpu_batches):
        load_weight(i, j+1, k)
        load_cache(i, j, k+1)
        store_hidden(i, j, k-1)
        load_hidden(i, j, k+1)
        compute_layer(i, j, k)
        store_cache(i, j, k-1)
        sync()
  # Epilogue
  ...

# h is the hidden states (activations)
def attention_forward(h, ...):
  # the read/write buffer are intermediate stops for prefetching
  if prefill:
    h, new_k_cache, new_v_cache = compute_attention(h, ...)
    # select K heavy hitters and K recent tokens
    new_k_cache, new_v_cache = select(new_k_cache, new_v_cache, K)
    cache_write_buffer.store(new_k_cache, new_v_cache)
  else:
    k_cache, v_cache = cache_read_buf.pop()
    # evict_ids track the entries that will be evicted
    h, new_k_cache, new_v_cache, evict_ids =
            compute_attention(h, k_cache, v_cache, ...)
```

```
    cache_write_buffer.store(new_k_cache, new_v_cache, evict_ids)
  return h

def store_cache(...):
  if prefill:
    # store cache directly
    ...
  else:
    k_new, v_new, evict_ids = cache_write_buffer.pop()
    # circular queue for the last K entries
    # extract the index for the oldest token at i-th iteration
    oldest = ((i - 1) % K) - K
    # update the KV cache (k_home and v_home)
    cache_replace(k_home, evict_ids, k_new, K, oldest)
    cache_replace(v_home, evict_ids, v_new, K, oldest)
```

# B Extended Related Works, Discussions, and Limitations

The goal of this section is to first introduce more background and related works for Section 2, then describe some previous attempts in our experiments as well as discuss the social impact and limitations of this work.

## B.1 Extended Related Works

**Quantization, Pruning, Distillation for Inference.** Previously, model compression algorithms have been extensively investigated as a viable approach for mitigating the computational resource requirements of model inference. These algorithms can be broadly categorized into three groups: (1) quantization [55, 56, 57, 58], which involves mapping model parameters or activations from high-precision data types to low-precision counterparts, such as using 8-bit integers instead of the commonly employed 32-bit floating point format; (2) pruning or sparsity [59, 60, 61, 62], which aims to eliminate unnecessary neurons or weights within the models; (3) and distillation [63, 64, 65, 66] where predictions from larger models are utilized as supervised information to train smaller models.

**Transformer in NLP.** Transformers [67] as a popular option have been frequently adopted by plenty of natural language processing (NLP) applications with prevailing successes [68, 69, 70, 71, 72, 46, 73, 13, 74, 75]. Roughly, modern transformer-based networks can be categorized into two groups: (1) Encoder-Decoder or Encoder-only (*i.e.*, BERT-style models [76]). This type of transformers commonly leverages the Masked Language Modeling task which encourages models to capture the intrinsic relationship between words and their context. Notable examples include BERT [76], RoBBERTa [69] and T5 [77]. (2) Decoder-only (*i.e.*, GPT-style models [78]). Usually, this group of transformers adopts the Casual Language Modeling task, which is optimized to generate the next word/token in a sequence based on the preceding words/tokens. Such an autoregressive manner is highly preferred by downstream tasks like text generation and question answering. GPT-3 [79], OPT [39], PaLM [13], and BLOOM [80] are representative architectures within this huge family.

**Training of Transformer.** Training a gigantic transformer-based model is not trivial. It notoriously suffers from various issues such as overfitting, instability, etc. [81, 82, 83] analyze these bottlenecks from the optimization perspective. To address the issues, a great amount of pioneering effort is devoted, including data augmentations [84, 85], a better initialization [86, 83, 87, 88], customized optimizers [89], improved normalization [90, 91], weight decay [92], and early stopping. However, there is still a long way to go before we can fully clarify the mystery of transformer training.

## B.2 Discussions and Limitations

**Previous Attempts.** During our experiments, we find several noteworthy observations. In $H_2O$, employing the accumulated attention score to evict KV embeddings can lead to a potential bias favoring the least recent tokens. This bias arises because most previous tokens have a higher number of attention scores, resulting in a higher accumulated attention score and, consequently, a greater likelihood of being retained. To address this concern, we conducted an additional experiment utilizing